LLM-Guided Grammar-Aware Fuzzing of PostgreSQL

Finding the bugs that do not crash

CS-5187 term project A3 · final presentation · 12 November 2026

The problem: silent wrong results

No crash, no error, no log line: the answer is just wrong.

```
SELECT c, '(,)'::int4range @> c AS contains
FROM (VALUES (NULL::int), (1)) v(c);

  c    | contains
 NULL  | t        <- wrong: must be NULL
 1     | t
```

Why it matters

- Users cannot notice it
- Present in released 17 and 18
- Only an oracle that knows the right answer sees it

20 of 23

recent PostgreSQL fixes change a query's result

9 of 20

visible to a crash-only fuzzer

Our idea

Keep every query as a typed tree, and let the server judge every LLM proposal.

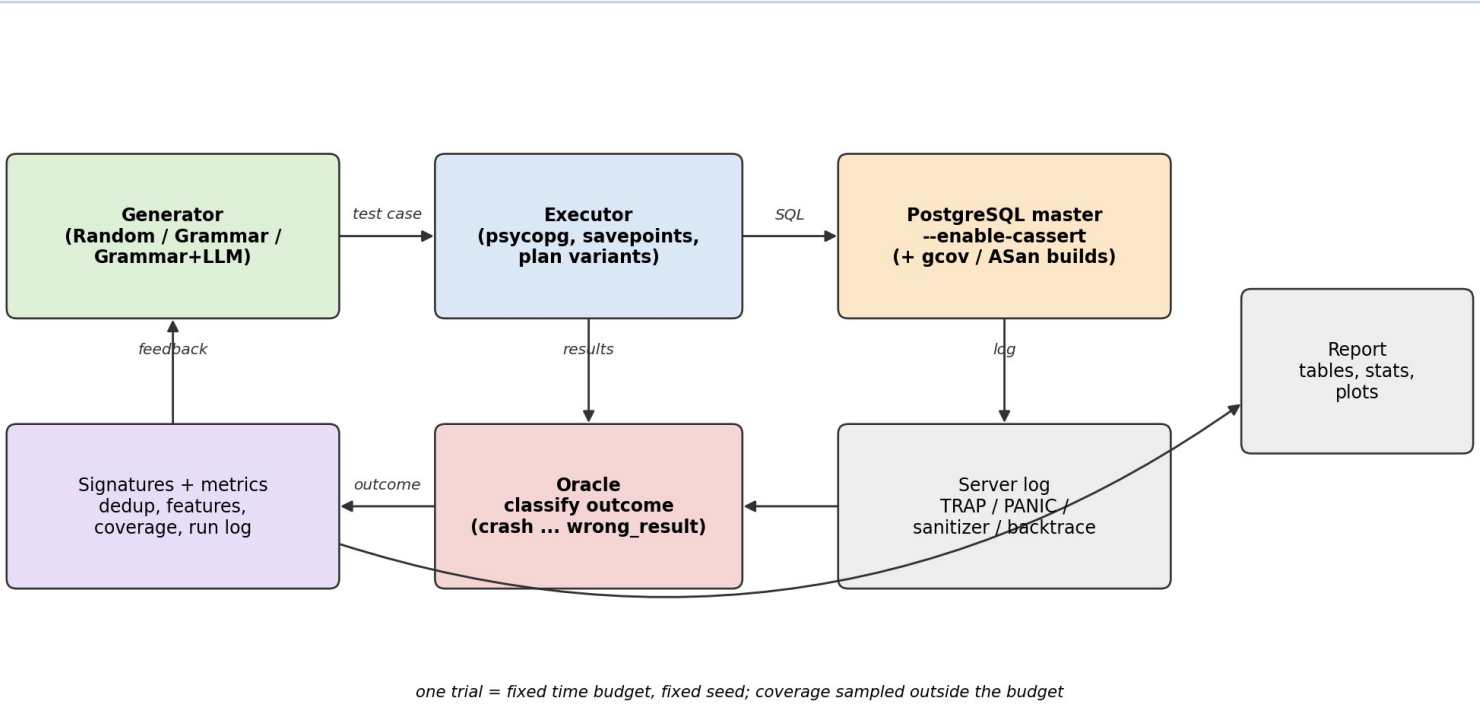
Approach	Strength	Gap
Random / mutation	fast	mostly invalid SQL
SQLsmith (grammar)	deep, typed	crashes only
SQLancer (TLP, NoREC)	wrong results	small generator
Fuzz4All (LLM)	new features	LLM on the hot path

Three claims

- One tree, four jobs: generate, mutate, rewrite, reduce
- Soundness we measure: validity and oracle precision
- The LLM proposes, the server decides

How it works

A fuzzing loop that runs thousands of test cases per minute, and an LLM loop that runs before a campaign.



Pipeline

- Generate from a typed grammar
- Attach oracle checks to the tree
- Run on the server, many ways
- Triage: reproduce, reduce, localize
- LLM extends the grammar

One tree, four jobs

SQL is generated as a derivation tree whose nodes know their type and scope.

The tree does four jobs

- Generate: derive a typed tree
- Mutate: re-derive one subtree
- Rewrite: an oracle changes one typed node
- Reduce: replace subtrees by their smallest derivation

78

grammar rules, 421 alternatives

13

SQL types

99%

of generated statements run

590 → 79

tokens after tree reduction

Oracles that know the right answer

Two queries that must agree, or one query under settings that must not change it.

Example: TLP

- Q = SELECT … WHERE p
- Q must equal Q(p) + Q(NOT p) + Q(p IS NULL)
- A difference means a bug

Oracle	Detects
Metamorphic rewrites (TLP, NoREC, expression)	wrong results
18 plan variants	plan-dependent results
Cross-version (master vs 18.6)	regressions
Row-level security check	policy leaks
CERT row estimates	estimate anomalies
Crash, assertion, error, hang	everything else

Triage: from failure to upstream report

Every candidate is reproduced three times, reduced, and localized.

```
bhuvi@BlazingBhuvi:~/cs5187-term-project/src$ cat ~/cs5187-term-project/runs/memberA-rq3/logs/A3_triage.txt
[cde9712bc8db] assertion via assertion: reproducing x3
  statements: 3 -> 2
  statements: 2 -> 1
  clauses: 572 -> 570 tokens
  clauses: 570 -> 543 tokens
  clauses: 543 -> 529 tokens
  clauses: 529 -> 527 tokens
  clauses: 527 -> 515 tokens
  items: 515 -> 420 tokens
  items: 420 -> 410 tokens
  items: 410 -> 318 tokens
  items: 318 -> 314 tokens
  items: 314 -> 290 tokens
  items: 290 -> 277 tokens
  items: 277 -> 273 tokens
  items: 273 -> 269 tokens
  items: 269 -> 265 tokens
  items: 265 -> 263 tokens
  items: 263 -> 261 tokens
  groups: 261 -> 201 tokens
  groups: 201 -> 162 tokens
  groups: 162 -> 141 tokens
  groups: 141 -> 139 tokens
  groups: 139 -> 125 tokens
  groups: 125 -> 116 tokens
  reduced to 1 statement(s), 116 tokens in the culprit (406 runs, 73.3 s)
1 candidate(s) triaged into ../results/triage_demo; verdict template: ../results/triage_demo/verdicts.template.yaml
bhuvi@BlazingBhuvi:~/cs5187-term-project/src$
```

```
bhuvi@BlazingBhuvi:/mnt/c/Users/Bhuva/OneDrive/Desktop/cs5187-term-project-main$ cat ~/cs5187-term-project/runs/memberA-rq3/triage_demo/cde9712bc8db/upstream_report
.txt 2>&1
Subject: fails an assertion -- <one line: the feature involved>

PostgreSQL version: 20devel (built with --enable-cassert)
Also checked on: <latest release: run the script there too>
Platform: <OS, compiler>

Description
-----------

The following script fails an assertion.

TRAP: failed Assert("list_length(rcexpr->inputcollids) == nopers"), File: "execExpr.c", Line: 2078

Script
------

-- pgfuzz triage of cde9712bc8db: assertion via assertion
SELECT CASE CASE WHEN NULL THEN NULL ELSE (SELECT MAX ('{NULL}'::text[]) FROM JSON_TABLE (CASE WHEN (ROW (10.0::numeric, 10) >= ROW (0.0::numeric, 1) IS FALSE) THEN
 '{"k": -1, "a": [1, 2, 3]}'::jsonb ELSE '{"k": 1, "a": [1, 2]}'::jsonb END, '$.a[*]' COLUMNS ( c1 int PATH '$')) AS a6) END WHEN NULL THEN NULL WHEN MAX ('{a,b,c,d
}'::text[]) OVER (PARTITION BY '{1,2,3}'::int[]) THEN NULL ELSE '{NULL,1}'::int[] END AS c0 FROM (SELECT a1.c0 AS c0 FROM t0 AS a1) AS a2;

Expected
--------

<what should happen, and why: SQL standard / documentation section>

<If bisected: first bad commit (scripts/bisect_pg.sh).>
bhuvi@BlazingBhuvi:/mnt/c/Users/Bhuva/OneDrive/Desktop/cs5187-term-project-main$
```

x3

reproduced on a fresh connection

590 → 79

tokens, tree then token reduction

localized

planner path or always-applied rewrite

The LLM extension and its gate

The LLM reads a recent commit; a proposal counts only if the server agrees.



```
gow@Gowtham:~/cs5187-term-project/runs/memberB-rq1-llm$ tail -40 logs/B2_extend.txt
| finished | 9 |
| accepted | 4 |
| acceptance_rate | 0.444 |
| accepted_fragments | 4 |
| accepted_by_round | {"1": 2, "2": 1, "3": 1} |
| rounds_total | 24 |
| rejections_by_stage | {"validity": 15, "parse": 2, "coverage/seeds": 1, "reachability": 2} |
| tokens | {"input_tokens": 312197, "output_tokens": 156417, "cache_read_input_tokens": 46568, "cache_creation_input_tokens": 0} |
| cost_usd | 0.0 |
| mean_validity | 0.376 |
| coverage_gain_lines | 34 |
| gate_bug_candidates | 0 |
| rewrites_proposed | 7 |
| rewrites_accepted | 7 |
| rewrites_with_mismatch | 0 |
18:30:10 INFO d4c15309e7 round 1: rejected
  d4c15309e7-r2.json: gemini-3-flash-preview, 15659 in / 14514 out, 73s

=== extend invocation 15 (18:31:22) ===
1 commit(s) finished, 0 accepted; ledger: ../results/llm/ledger.jsonl
| metric | value |
|---|---|
| commits | 10 |
| finished | 10 |
| accepted | 4 |
| acceptance_rate | 0.4 |
| accepted_fragments | 4 |
| accepted_by_round | {"1": 2, "2": 1, "3": 1} |
| rounds_total | 25 |
| rejections_by_stage | {"validity": 16, "parse": 2, "coverage/seeds": 1, "reachability": 2} |
| tokens | {"input_tokens": 327856, "output_tokens": 170931, "cache_read_input_tokens": 46568, "cache_creation_input_tokens": 0} |
| cost_usd | 0.0 |
| mean_validity | 0.365 |
| coverage_gain_lines | 34 |
| gate_bug_candidates | 0 |
| rewrites_proposed | 7 |
| rewrites_accepted | 7 |
| rewrites_with_mismatch | 0 |
18:32:01 INFO d4c15309e7 round 2: rejected
=== extend finished: no prompt left ===
```

Gate checks

- Parses
- Used at least 5 times
- 60% of statements run
- Seeds run
- More changed lines covered

Outcome

- 10 commits, 4 accepted
- +5 SQL features
- Found BUG #19803
- One rewrite slipped through: now rejected

Evaluation

Four questions, seven configurations, 5 trials of one hour each.

Question	Measure
RQ1 Reach	validity, coverage, SQL features, plan shapes
RQ2 Bugs	bugs found and oracle precision
RQ3 Ground truth	23 re-injected real bugs found
RQ4 LLM	what the extension adds and costs

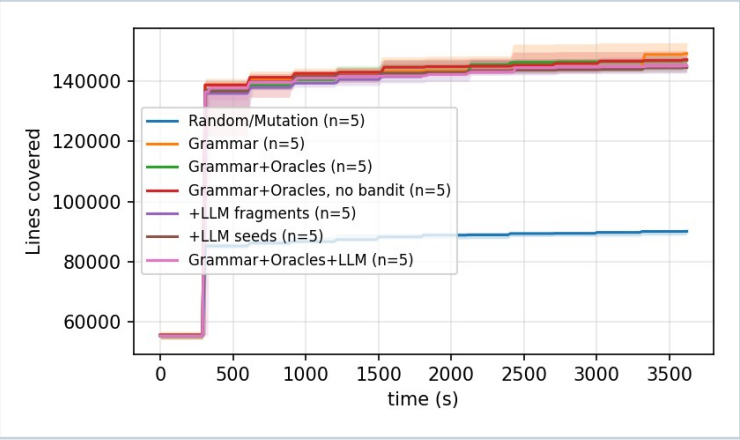
Setup

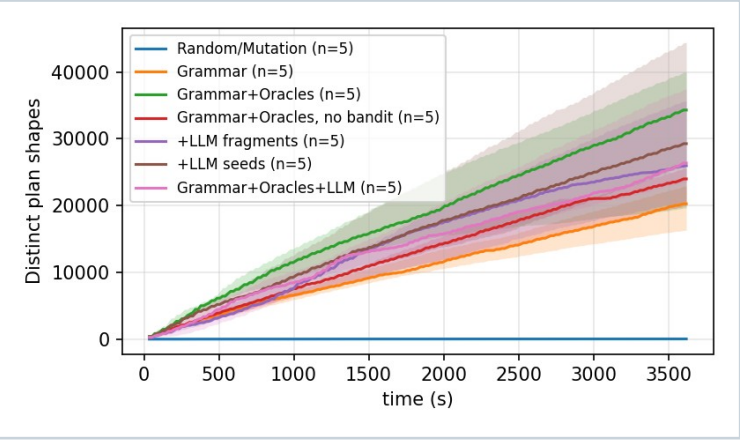
- Master 26bd08fb, assertions on
- 7 arms x 5 trials x 1 h
- Two laptops, one per experiment
- Same seeds for all arms

RQ1: the typed grammar reaches much more

Random mutation runs 15 times more tests and still reaches less.

Arm	Valid	Lines	Features	Plan shapes
Random / mutation	**26.0%**	22.2%	23	43
Grammar	**99.3%**	**36.7%**	101	20,279
Grammar + oracles	96.3%	36.2%	101	**34,313**
+ LLM	95.3%	35.7%	106	26,375





Reading

- Grammar vs random: A12 = 1.00, p = 0.008
- Oracles: plan shapes x1.7
- LLM: +5 features, no extra lines

RQ2: nine bugs reported upstream

Two confirmed by committers, two already fixed, one fix written by our team.

Bug	Report	Oracle	State
Parallel Append assertion	#19803	TLP (LLM arm)	**fixed**
to_char(NaN) breaks NORMALIZE	#19771	assertion	**fixed**
MERGE: SubPlan with no parent	#19802	internal error	**confirmed**
IGNORE NULLS frame never returns	pgsql-bugs	hang	**answered**
range @> NULL folded to a constant	#19793	(SELECT e) rewrite	**reported, patch**
JSON_TABLE assigns no collations	#19768	assertion	**reported, patch**
NOT IN anti-join ignores a cast	pgsql-bugs	TLP	**reported, patch**
duplicate statistics expression	pgsql-bugs	assertion	**reported, patch**
non-finite imported MCV frequencies	pgsql-hackers	CERT estimate	**patch**

The tool finds every bug we reported

One 150-second run on the base build reports all nine.

9 / 9

reported bugs rediscovered

150 s

one trial, base build

99.3%

validity unchanged

What we added

- Seeds that carry their own oracle (the query that must agree)
- Grammar: statistics on expressions, statistics import, user-defined casts, IGNORE NULLS frames
- A hang oracle: a statement that ignores its timeout is reported, the server restarted

RQ2: precision is the bottleneck

Every one of 93 candidates has a verdict.

Verdict	Count
real, new	8
real, reported first by others	15
false positive	42
flaky	28
open	0

0.16

precision

0.35

if duplicates count

Eleven false-positive classes, all fixed

- Equal intervals printed differently: 16
- RLS rewrite missed a repeated sub-select: 5
- Cross-version state divergence: 5
- ORDER BY NULL in ordered aggregates: 2
- LLM rewrite that changed the collation: 1
- Six more found during development

RQ3: re-injected real bugs

One hour finds few of them; we say so.

```
bhuvi@BlazingBhuvi:~/cs5187-term-project/src$ python3 -m pgfuzz bench verify configs/rq3.yaml
| bug | commit | symptom | all injected | evidence |
|---|---|---|---|---|
| rb-11b56ee2 | 11b56ee2 | assertion | assertion | pathkeys.c: var->varno == rel->relid |
| rb-94e1385e | 94e1385e | wrong_result | wrong_result | stmt 0: 1 rows on fixed, 1 on buggy |
| rb-41180127 | 41180127 | missing_error | missing_error | stmt 0: fixed raises 22030 duplicate JSON object key value |
| rb-b28a2b2b | b28a2b2b | wrong_result | wrong_result | stmt 5: 1 rows on fixed, 1 on buggy |
| rb-f5635c27 | f5635c27 | none | none |  |
| rb-d8444163 | d8444163 | wrong_result | wrong_result | stmt 3: 0 rows on fixed, 1 on buggy |
| rb-62e46c00 | 62e46c00 | error | error | stmt 2: 42P10 there is no unique or exclusion constraint matching the ON CONFLICT specific |
| rb-738d19d4 | 738d19d4 | internal_error | internal_error | stmt 6: XX000 unsupported join alias expression |
| rb-6b858a36 | 6b858a36 | assertion | assertion | subselect.c: !IsA(node, SubPlan) |
| rb-8b29b8da | 8b29b8da | internal_error | internal_error | stmt 10: XX000 could not find pathkey item to sort |
| rb-d764d61a | d764d61a | wrong_result | wrong_result | stmt 7: 1 rows on fixed, 1 on buggy |
| rb-dfcf6fa7 | dfcf6fa7 | internal_error | internal_error | stmt 6: XX000 could not find pathkey item to sort |
| rb-a464d31f | a464d31f | assertion | assertion | relnode.c: join_clause_is_movable_into(rinfo, baserel->relids, joinrelids) |
| rb-852fd5b8 | 852fd5b8 | internal_error | internal_error | stmt 5: XX000 no relation entry for relid 3 |
| rb-a8bb1a77 | a8bb1a77 | assertion | assertion | rewriteManip.c: context->new_index != INVALID_VAR |
| rb-9d0adee8 | 9d0adee8 | none | none |  |
| rb-895c1b5f | 895c1b5f | none | none |  |
| rb-35d3f206 | 35d3f206 | internal_error | internal_error | stmt 21: XX000 variable not found in subplan target list |
| rb-1bf74c80 | 1bf74c80 | wrong_result | wrong_result | stmt 4: 4 rows on fixed, 3 on buggy |
| rb-7774249d | 7774249d | wrong_result | wrong_result | stmt 1: 1 rows on fixed, 1 on buggy |
| rb-0d995793 | 0d995793 | wrong_result | wrong_result | stmt 4: 0 rows on fixed, 1 on buggy |
| rb-d6eac691 | d6eac691 | wrong_result | wrong_result | stmt 0: 0 rows on fixed, 2 on buggy |
| rb-3b120b1e | 3b120b1e | missing_error | missing_error | stmt 0: fixed raises 22003 integer out of range |
note: rb-94e1385e raises errors on the fixed server: stmt 1: 2201E cannot take logarithm using base one; stmt 2: 2201E cannot take logarithm using base one
note: rb-41180127 raises errors on the fixed server: stmt 0: 22030 duplicate JSON object key value
note: rb-6b858a36 raises errors on the fixed server: stmt 3: 21000 more than one row returned by a subquery used as an expression
note: rb-3b120b1e raises errors on the fixed server: stmt 0: 22003 integer out of range; stmt 1: 22003 integer out of range; stmt 2: 22003 integer out of range

20/23 bugs observable; copy the symptoms into /home/bhuvi/cs5187-term-project/src/bench/reinjected/bugs.yaml
bhuvi@BlazingBhuvi:~/cs5187-term-project/src$
```

Arm	Bugs found (of 20)
Random	0
Grammar	1
Grammar + oracles	2
+ LLM	2

Honest reading

- One hour and 5 trials are short for bugs that need feature combinations; the oracle advantage shows in RQ2, not here

What we learned

Three findings.

Typing pays

- 99% vs 26% valid
- 36.7% vs 22.2% of lines
- random runs 15x more tests

Precision is the bottleneck

- real wrong results in 17 and 18
- 11 false-positive classes
- each needed a soundness fix

The LLM needs a gate

- +5 features, no extra coverage
- 4 of 10 commits usable
- tested is not proven

Limitations

What our results do not show.

Statistics

- 5 trials per arm: effect sizes, not always significance

Measurement

- False-positive fixes not re-measured by a new campaign

LLM

- Free-tier Gemini Flash, 10 commits

Scope

- One system, one schema, one commit; rediscovery uses seeds

Conclusion

A fuzzer for the bugs that do not crash.

9
bugs reported upstream

2
fixed on master

36.7%
backend lines (random 22.2%)

151
unit tests

Take-away

Generating valid SQL is the easy part; knowing the right answer is the hard part. Typed trees make sound rewrites possible, and measuring precision tells you where the oracle is still wrong.

Thank you. Questions?

github.com/PASS-IITG/cs5187-term-project-blazing-bhuvi